Desafio Técnico — Conexão Python + SQL Server com pyodbc

Contexto do problema

A PetVida, uma clínica veterinária de bairro, controla tutores, pets e atendimentos em planilhas. Com o crescimento, surgiram erros: consultas lançadas para pets que não existem mais, procedimentos cobrados com valor errado e tutores duplicados.

A clínica contratou sua equipe para construir a camada de acesso a dados em Python de um novo sistema. O banco de dados no SQL Server já foi modelado e está pronto. Falta o código que conecta a aplicação ao banco e executa as operações de cadastro, consulta, alteração e exclusão (CRUD) com segurança.

O foco do desafio não é a interface nem a modelagem: é a conexão. Uma conexão mal feita deixa, abre brecha para SQL Injection, grava metade de uma operação quando algo falha e deixa conexões abertas no servidor. Seu código precisa evitar todos esses problemas.

Objetivos de aprendizagem

Ao concluir o desafio, o aluno será capaz de:

1. Configurar o driver ODBC e montar uma *connection string* para SQL Server com autenticação Windows ou SQL, sem credenciais no código.
2. Executar comandos parametrizados com o marcador ? e demonstrar que eles impedem SQL Injection.
3. Recuperar chaves geradas por IDENTITY com OUTPUT INSERTED.
4. Navegar relacionamentos 1:N e N:M com JOIN e manter a tabela associativa.
5. Traduzir erros do banco (pyodbc.IntegrityError, OperationalError) em exceções de negócio compreensíveis.

Modelo de dados entregue

O banco PetVida é entregue pronto no script 01_estrutura_petvida.sql, com 6 tabelas, restrições e dados iniciais. O aluno não altera a estrutura; apenas a consome a partir do Python.

Um tutor tem vários pets (1:N); cada consulta pertence a um pet e a um veterinário (1:N); consultas e procedimentos formam um N:M resolvido por consulta_procedimento.

Tabela	Papel no desafio	Regras que o código deve respeitar
tutor	Lado 1 de tutor → pet	CPF único com 11 dígitos; exclusão bloqueada se houver pets
pet	Lado N de tutor; lado 1 de consulta	Espécie em lista fechada; peso positivo; exclusão bloqueada se houver consultas
veterinario	Lado 1 de consulta	CRMV único
procedimento	Catálogo; um lado do N:M	Descrição única; preço base ≥ 0
consulta	Lado N de pet e veterinário; outro lado do N:M	Status AGENDADA, REALIZADA ou CANCELADA
consulta_procedimento	Tabela associativa do N:M	PK composta; quantidade > 0; itens apagados em cascata com a consulta

Dados iniciais: 4 tutores (Roberto sem pets, para testar exclusão), 6 pets, 3 veterinários, 7 procedimentos, 5 consultas e 8 itens de consulta.

Preparação do ambiente

1. Execute 01_estrutura_petvida.sql como administrador. Ao final, a consulta de conferência deve listar 5 consultas.
2. Instale o ODBC Driver (pyodbc).
    a. Em um ambiente virtual: pip install pyodbc.

Etapas do desafio

As etapas são incrementais: cada uma usa a conexão construída na Etapa 1.

Etapa 1 — Conexão (arquivo db.py)

É o núcleo do desafio. Implemente:

- montar_connection_string(): monta a conexão e retorna.
- conectar(): pyodbc.connect(..., autocommit=False).
- testar_conexao(): retorna servidor, banco e versão do SQL Server.

Exemplo do uso esperado:

```python
with transacao() as cur:
    cur.execute("SELECT nome FROM dbo.tutor WHERE id_tutor = ?", 1)
    print(cur.fetchone().nome)
```

Etapa 2 — CRUD de tutor (repositorio.py) · 21 pontos

- Create: criar_tutor() com OUTPUT INSERTED.id_tutor, devolvendo o id gerado.
- Read: buscar_tutor() e listar_tutores(filtro_nome) com LIKE parametrizado.
- Update: atualizar_tutor(id, **campos) aceitando só nome, email e telefone.
- Delete: excluir_tutor().
- CPF duplicado ou inválido apresenta erro.

Etapa 3 — Relacionamento 1:N tutor → pet

- criar_pet(), atualizar_peso_pet(), transferir_pet() e excluir_pet().
- listar_pets_do_tutor(): navega do lado 1 para o lado N.
- buscar_pet_com_tutor(): navega do lado N para o lado 1 com JOIN.

Etapa 4 — Relacionamento N:N consulta ↔ procedimento

- agendar_consulta(id_pet, id_vet, data_hora, motivo, procedimentos): grava a consulta e seus itens. O valor cobrado é o preco_base atual. Se um item falhar, nada é gravado.
- adicionar_procedimento() e remover_procedimento(): mantêm a tabela associativa; item repetido ou consulta cancelada geram erro.
- listar_procedimentos_da_consulta() e listar_consultas_por_procedimento(): o N:M navegado nos dois sentidos.
- total_da_consulta(), atualizar_status_consulta() e excluir_consulta() (itens somem pelo ON DELETE CASCADE).

Bônus

- historico_do_pet() usando a view vw_resumo_consulta.
- Um menu de terminal (main.py) que use o repositório.

Regras técnicas obrigatórias

O descumprimento de qualquer regra abaixo zera a etapa correspondente, mesmo que o verificador aponte OK.

Regra	Correto	Proibido
Parametrização	cur.execute("... WHERE cpf = ?", cpf)	f"... WHERE cpf = '{cpf}'" ou + com dados do usuário
Recursos	Conexão fechada no finally / with	Conexão global aberta durante todo o programa
Estrutura do banco	Usar o script entregue sem alterações	Remover constraints para "fazer passar"

Nomes de colunas não podem ser parametrizados com ?. Quando o SQL precisar ser montado dinamicamente (como em atualizar_tutor), os nomes devem vir de uma lista branca definida no código.

Entrega e critérios de avaliação

Entregue um repositório (ou .zip) com db.py, repositorio.py, README.md e um print do verificador. O README explica como configurar as variáveis e rodar o projeto. Prazo e forma de envio seguem o AVA da disciplina.

Recursos didáticos e referências

- MICROSOFT. *Python SQL driver – pyodbc*. Microsoft Learn. Disponível em: learn.microsoft.com/sql/connect/python/pyodbc. Acesso em: 14 out. 2026.
- MICROSOFT. *Download ODBC Driver for SQL Server*. Microsoft Learn. Disponível em: learn.microsoft.com/sql/connect/odbc. Acesso em: 14 out. 2026.
- MICROSOFT. *OUTPUT clause (Transact-SQL)*. Microsoft Learn. Disponível em: learn.microsoft.com/sql/t-sql/queries/output-clause-transact-sql. Acesso em: 14 out. 2026.
- MKLEEHAMMER. *pyodbc Wiki*. GitHub. Disponível em: github.com/mkleehammer/pyodbc/wiki. Acesso em: 14 out. 2026.
- OWASP. *SQL Injection Prevention Cheat Sheet*. Disponível em: cheatsheetseries.owasp.org. Acesso em: 14 out. 2026.
- PYTHON SOFTWARE FOUNDATION. *PEP 249 – Python Database API Specification v2.0*. Disponível em: peps.python.org/pep-0249. Acesso em: 14 out. 2026.